

PROBLEM 1: DESIGN PARKING SYSTEM

Platform: LeetCode

Problem: 1603. Design Parking System

Aim:

To design a parking system that maintains the number of available spaces for big, medium, and small cars.

Concepts Used:

- Class
- Object
- Constructor
- Instance variables
- Methods

Example:

Input:

ParkingSystem(1, 1, 0)

addCar(1)

addCar(2)

addCar(3)

addCar(1)

Output:

true

true

false

false

Approach:

Create a ParkingSystem class.

Store the available number of:

- Big spaces
- Medium spaces
- Small spaces

The addCar() method checks whether a suitable space is available.

If available, decrease the corresponding count and return true.

Otherwise, return false.

JAVA CODE

```
class ParkingSystem {  
    int big;  
    int medium;  
    int small;
```

```

public ParkingSystem(int big, int medium, int small) {
    this.big = big;
    this.medium = medium;
    this.small = small;
}

```

```

public boolean addCar(int carType) {

```

```

    if (carType == 1) {

```

```

        if (big > 0) {
            big--;
            return true;
        }

```

```

    } else if (carType == 2) {

```

```

        if (medium > 0) {
            medium--;
            return true;
        }

```

```

    } else {

```

```

        if (small > 0) {
            small--;
            return true;
        }
    }

```

```

    return false;
}

```

```

}

```

Complexity:

Time: O(1) per operation

Space: O(1)

PROBLEM 2: JAVA INSTANCEOF KEYWORD

Platform: HackerRank

Problem: Java Instanceof Keyword

Aim:

To understand inheritance and determine whether an object belongs to a particular class using the instanceof operator.

Syntax:

object instanceof ClassName

The result is either true or false.

JAVA CODE

```
import java.util.*;
class Student {
}
class Rockstar {
}
class Hacker {
}
public class InstanceOFTutorial {
    static String count(ArrayList<Object> mylist) {

        int students = 0;
        int rockstars = 0;
        int hackers = 0;

        for (Object obj : mylist) {

            if (obj instanceof Student) {
                students++;
            }

            if (obj instanceof Rockstar) {
                rockstars++;
            }

            if (obj instanceof Hacker) {
                hackers++;
            }
        }

        return students + " " + rockstars + " " + hackers;
    }
}
```

}

Explanation:

The instanceof operator checks the type of an object.

Example:

obj instanceof Student

returns true if obj is an instance of Student.

Complexity:

Time: $O(n)$

Space: $O(1)$, excluding the input collection

=====

SESSION 2

CLASS-TO-CLASS INTERACTION

OBJECTIVE

To understand how different classes interact with each other using object references and relationships.

Important Relationships:

- Association
- Aggregation
- Composition

Example:

Teacher ---- Student

A class can contain a reference to another class.

PROBLEM 3: DESIGN UNDERGROUND SYSTEM

Platform: LeetCode

Problem: 1396. Design Underground System

Aim:

To design a system that records passengers checking into and checking out of stations and calculates the average travel time.

Concepts Used:

- Classes
- Objects
- HashMap
- Class-to-class interaction

Approach:

Use one HashMap to store current check-ins.

Store:

Customer ID

Station

Check-in time

Use another HashMap to store:

Route

Total travel time

Number of trips

JAVA CODE

```
import java.util.*;
```

```
class UndergroundSystem {
```

```
    class CheckIn {
```

```
        String station;
```

```
        int time;
```

```
        CheckIn(String station, int time) {
```

```
            this.station = station;
```

```
            this.time = time;
```

```
        }
```

```
    }
```

```
    class Route {
```

```
        int totalTime;
```

```
        int count;
```

```
        Route(int totalTime, int count) {
```

```
            this.totalTime = totalTime;
```

```
            this.count = count;
```

```
        }
```

```
    }
```

```
    HashMap<Integer, CheckIn> checkIns;
```

```
    HashMap<String, Route> routes;
```

```
    public UndergroundSystem() {
```

```
        checkIns = new HashMap<>();
```

```
        routes = new HashMap<>();
```

```
    }
```

```

public void checkIn(int id, String stationName, int t) {

    checkIns.put(id, new CheckIn(stationName, t));
}

public void checkOut(int id, String stationName, int t) {

    CheckIn checkIn = checkIns.get(id);

    String key = checkIn.station + "->" + stationName;

    int travelTime = t - checkIn.time;

    if (!routes.containsKey(key)) {

        routes.put(key, new Route(travelTime, 1));

    } else {

        Route route = routes.get(key);

        route.totalTime += travelTime;
        route.count++;
    }

    checkIns.remove(id);
}

public double getAverageTime(
    String startStation,
    String endStation) {

    String key = startStation + "->" + endStation;

    Route route = routes.get(key);

    return (double) route.totalTime / route.count;
}

```

```
}
```

Complexity:

checkIn: $O(1)$ average

checkOut: $O(1)$ average

getAverageTime: $O(1)$ average

Space: $O(n)$

PROBLEM 4: DESIGN BROWSER HISTORY

Platform: LeetCode

Problem: 1472. Design Browser History

Aim:

To design a browser history system that supports:

- Visit
- Back
- Forward

Approach:

Use a doubly linked list.

Each node contains:

URL

Previous node

Next node

JAVA CODE

```
class BrowserHistory {  
    class Node {
```

```
        String url;  
        Node prev;  
        Node next;
```

```
        Node(String url) {  
            this.url = url;  
        }  
    }
```

```
}
```

```
    Node current;
```

```
    public BrowserHistory(String homepage) {  
  
        current = new Node(homepage);  
    }
```

```

public void visit(String url) {

    Node newNode = new Node(url);

    current.next = newNode;
    newNode.prev = current;

    current = newNode;
}

public String back(int steps) {

    while (steps > 0 && current.prev != null) {

        current = current.prev;
        steps--;
    }

    return current.url;
}

public String forward(int steps) {

    while (steps > 0 && current.next != null) {

        current = current.next;
        steps--;
    }

    return current.url;
}

```

```

}

```

Complexity:

Visit: $O(1)$

Back: $O(\text{steps})$

Forward: $O(\text{steps})$

Space: $O(n)$

=====

SESSION 3

CLASS-TO-METHOD INTERACTION

OBJECTIVE

To understand how methods operate on the data stored inside objects.

A class generally contains:

Data + Methods

PROBLEM 5: DESIGN HASHSET

Platform: LeetCode

Problem: 705. Design HashSet

Aim:

To design a HashSet without using Java's built-in HashSet class.

Required Operations:

- add(key)
- remove(key)
- contains(key)

Approach:

For the given key range, use a boolean array.

true = key exists

false = key does not exist

JAVA CODE

```
class MyHashSet {
    boolean[] set;

    public MyHashSet() {

        set = new boolean[1000001];
    }

    public void add(int key) {

        set[key] = true;
    }

    public void remove(int key) {

        set[key] = false;
    }
}
```

```
public boolean contains(int key) {  
  
    return set[key];  
}
```

```
}
```

Complexity:

Add: $O(1)$

Remove: $O(1)$

Contains: $O(1)$

Space: $O(k)$

PROBLEM 6: GRADING STUDENTS

Platform: HackerRank

Problem: Grading Students

Aim:

To round student grades according to the given rules.

Rules:

If the grade is less than 38, do not round it.

Find the next multiple of 5.

If the difference between the grade and the next multiple is less than 3, round the grade.

Otherwise, keep the original grade.

Example:

Grade = 84

Next multiple = 85

Difference = 1

Result = 85

JAVA CODE

```
import java.util.*;
```

```
class Result {
```

```
public static List<Integer> gradingStudents(  
    List<Integer> grades) {
```

```
    List<Integer> result = new ArrayList<>();
```

```
    for (int grade : grades) {
```

```
        if (grade < 38) {
```

```

        result.add(grade);

    } else {

        int nextMultiple =
            ((grade / 5) + 1) * 5;

        if (nextMultiple - grade < 3) {
            result.add(nextMultiple);
        } else {
            result.add(grade);
        }
    }
}

return result;
}

```

}

Complexity:

Time: $O(n)$

Space: $O(n)$ for the result list

=====

SESSION 4

INHERITANCE

OBJECTIVE

To understand inheritance and how a child class can reuse properties and methods of a parent class.

Syntax:

class Child extends Parent

Parent class = Superclass

Child class = Subclass

PROBLEM 7: JAVA INHERITANCE 1

Platform: HackerRank

Problem: Java Inheritance 1

Aim:

To demonstrate basic inheritance in Java.

JAVA CODE

```

class Animal {
void walk() {
    System.out.println("I am walking");
}

}

class Bird extends Animal {
void fly() {
    System.out.println("I am flying");
}

void sing() {
    System.out.println("I am singing");
}

}

public class Solution {
public static void main(String[] args) {

    Bird bird = new Bird();

    bird.walk();
    bird.fly();
    bird.sing();
}

}

```

Output:

I am walking

I am flying

I am singing

Explanation:

Bird extends Animal.

Therefore, a Bird object can use:

walk() - inherited from Animal

fly() - defined in Bird

sing() - defined in Bird

Complexity:

Time: $O(1)$

Space: $O(1)$

PROBLEM 8: JAVA INHERITANCE 2

Platform: HackerRank

Problem: Java Inheritance 2

Aim:

To demonstrate inheritance and the use of inherited methods.

JAVA CODE

```
class Arithmetic {
    public int add(int a, int b) {
        return a + b;
    }
}

class Adder extends Arithmetic {
}

public class Solution {
    public static void main(String[] args) {

        Adder a = new Adder();

        System.out.println(
            "My superclass is: " +
            a.getClass().getSuperclass().getName()
        );

        System.out.println(
            a.add(10, 32) + " " +
            a.add(10, 3) + " " +
            a.add(10, 10)
        );
    }
}

Output:
My superclass is: Arithmetic
```

42 13 20

Explanation:

Adder extends Arithmetic.

Therefore, Adder inherits the add() method from Arithmetic.

Complexity:

Time: O(1)

Space: O(1)

=====

SESSION 5

DESIGNING REUSABLE CLASSES

OBJECTIVE

To understand how reusable classes and methods can be designed using Java Generics and object-oriented structures.

PROBLEM 9: JAVA GENERICS

Platform: HackerRank

Problem: Java Generics

Aim:

To understand how Java Generics allow the same method to work with different data types.

Generic method syntax:

```
public <T> void printArray(T[] array)
```

JAVA CODE

```
import java.util.*;  
public class Printer {  
    public <T> void printArray(T[] array) {
```

```
        for (T element : array) {
```

```
            System.out.println(element);
```

```
        }
```

```
    }
```

```
}
```

Example:

```
Integer[] intArray = {1, 2, 3};
```

```
String[] stringArray = {
```

```
"Hello",
```

```
"World"
```

```
};
```

Explanation:

<T> is a type parameter.

The same method can work with different types such as:

Integer

String

Double

Character

Complexity:

Time: $O(n)$

Extra Space: $O(1)$

PROBLEM 10: THRONE INHERITANCE

Platform: LeetCode

Problem: 1600. Throne Inheritance

Aim:

To design a system that maintains the order of succession in a royal family.

Operations:

- Birth
- Death
- Get inheritance order

Concepts Used:

- Classes
- Objects
- HashMap
- HashSet
- ArrayList
- Tree
- DFS

Approach:

Represent every person as a node in a family tree.

Store:

Person -> List of children

Maintain a HashSet containing people who have died.

To generate the inheritance order:

Start with the king.

Add the person if they are alive.

Visit each child in birth order.

Continue recursively.

JAVA CODE

```

import java.util.*;
class ThroneInheritance {
    String king;

    HashMap<String, List<String>> children;

    HashSet<String> dead;

    public ThroneInheritance(String kingName) {

        king = kingName;

        children = new HashMap<>();
        dead = new HashSet<>();

        children.put(
            kingName,
            new ArrayList<>()
        );
    }

    public void birth(
        String parentName,
        String childName) {

        children
            .computeIfAbsent(
                parentName,
                k -> new ArrayList<>()
            )
            .add(childName);

        children.putIfAbsent(
            childName,
            new ArrayList<>()
        );
    }

    public void death(String name) {

```



```

        dead.add(name);
    }

    public List<String> getInheritanceOrder() {

        List<String> result =
            new ArrayList<>();

        dfs(king, result);

        return result;
    }

    private void dfs(
        String person,
        List<String> result) {

        if (!dead.contains(person)) {

            result.add(person);
        }

        for (String child :
            children.get(person)) {

            dfs(child, result);
        }
    }
}

```

}

Complexity:

Birth: $O(1)$ average

Death: $O(1)$ average

Get Inheritance Order: $O(n)$

Space: $O(n)$

=====

QUICK REVISION TABLE

Design Parking System

Concept: Classes and Objects

Time: $O(1)$

Space: $O(1)$

Java Instanceof Keyword

Concept: Object Type Checking

Time: $O(n)$

Space: $O(1)$

Design Underground System

Concept: Class Interaction + HashMap

Time: $O(1)$ average

Space: $O(n)$

Design Browser History

Concept: Doubly Linked List + Objects

Time: $O(\text{steps})$

Space: $O(n)$

Design HashSet

Concept: Class Design

Time: $O(1)$

Space: $O(k)$

Grading Students

Concept: Methods + ArrayList

Time: $O(n)$

Space: $O(n)$

Java Inheritance 1

Concept: Inheritance

Time: $O(1)$

Space: $O(1)$

Java Inheritance 2

Concept: Inheritance

Time: $O(1)$

Space: $O(1)$

Java Generics

Concept: Generic Methods

Time: $O(n)$

Space: $O(1)$

Throne Inheritance

Concept: Tree + DFS + OOP

Time: $O(n)$

Space: $O(n)$

=====

IMPORTANT JAVA OOP CONCEPTS

CLASS

A class is a blueprint for creating objects.

Example:

```
class Student {  
    String name;  
    int age;
```

```
}
```

OBJECT

An object is an instance of a class.

Example:

```
Student s = new Student();
```

ENCAPSULATION

Encapsulation means combining data and methods inside a class and controlling access to the data.

Example:

```
class Student {  
    private int age;
```

```
    public void setAge(int age) {  
        this.age = age;  
    }
```

```
    public int getAge() {  
        return age;  
    }
```

```
}
```

INHERITANCE

Inheritance allows a child class to reuse functionality from a parent class.

Example:

```
class Dog extends Animal {  
}
```

POLYMORPHISM

Polymorphism allows the same method or interface to behave differently depending on the object.

Example:

```
Animal a = new Dog();
```

ABSTRACTION

Abstraction hides implementation details and exposes only the required functionality.

It can be implemented using:

- Abstract classes
- Interfaces

ASSOCIATION

Association represents a relationship between two independent classes.

Example:

Teacher ---- Student

AGGREGATION

Aggregation represents a weak "has-a" relationship.

The contained object can exist independently.

Example:

Department ---- Teacher

COMPOSITION

Composition represents a strong "has-a" relationship.

Example:

House ---- Room

GENERIC

Generics allow classes and methods to work with different data types.

Example:

```
class Box<T> {
```

```
    T value;
```

```
}
```

=====

FINAL EXAM CHECKLIST

Before the examination, make sure you can explain:

- ☐ What is a class?
- ☐ What is an object?
- ☐ Difference between class and object
- ☐ What is encapsulation?
- ☐ What is inheritance?
- ☐ What is polymorphism?
- ☐ What is abstraction?
- ☐ Difference between association, aggregation, and composition
- ☐ How instanceof works
- ☐ How classes interact with each other
- ☐ How objects interact through methods

- How to design reusable classes
- What are Java Generics?
- How to use a generic method
- How inheritance works using extends
- How to implement a tree using classes
- How DFS is used in object-oriented problems

=====

UNIT III SUMMARY

Object-Oriented Problem Solving combines DSA and OOP concepts.

The general problem-solving process is:

Identify Objects

↓

Create Classes

↓

Define Data

↓

Define Methods

↓

Establish Class Relationships

↓

Apply Data Structures

↓

Solve the Problem

=====

END OF UNIT III

Object-Oriented Problem Solving

Week 7 | Sessions 1–5 | 10 Hands-On Problems

You said:

PDF FRIENDLY

ChatGPT said:

Yes — PDF-friendly means no dark code boxes, no Markdown styling, and no colored background.

Use this exact setup when pasting into Word / Google Docs:

PDF Page Settings

Page: A4

Orientation: Portrait

Margins: Normal (1 inch)

Background: White

Body font: Times New Roman, 12 pt

Main headings: 18 pt, Bold, Black

Subheadings: 14 pt, Bold, Black

Code: Consolas, 10 pt, Black text

Code background: White

Line spacing: 1.15

Paragraph spacing: 6 pt

Page numbers: Bottom center

Important

Do not copy the code from a dark code block. Copy the plain-text document version above into Word/Google Docs, then apply the settings above.

For the cleanest PDF, use:

Word:

File → Save As → PDF

Google Docs:

File → Download → PDF Document (.pdf)

If you want, I can also give you a fully page-by-page A4 layout, with each problem starting cleanly on the appropriate page so you can paste it directly into Word and export without any black background.

You said:

GIVE AGAIN

ChatGPT said:

Yes. Here is the PDF-FRIENDLY VERSION AGAIN — designed for white A4 pages, with black text only and no dark code backgrounds.

Copy everything below into Microsoft Word or Google Docs, then export as PDF.

UNIT III – OBJECT-ORIENTED PROBLEM SOLVING

Course Outcome:

Design object-oriented structures for problem solving.

WEEK 7

=====

SESSION 1

INTRODUCTION TO OBJECT MODELING IN ALGORITHMIC PROBLEMS

=====

OBJECTIVE

To understand how real-world entities can be represented using classes and objects and how object-oriented structures can be used to solve algorithmic problems.

Important Concepts:

- Classes
- Objects
- Constructors
- Instance variables
- Methods

- Object modeling

PROBLEM 1: DESIGN PARKING SYSTEM

Platform: LeetCode

Problem Number: 1603

Aim:

To design a parking system that maintains the number of available spaces for big, medium, and small cars.

Vehicle Types:

Big

Medium

Small

Example:

Input:

ParkingSystem(1, 1, 0)

addCar(1)

addCar(2)

addCar(3)

addCar(1)

Output:

true

true

false

false

Approach:

Create a ParkingSystem class.

Store the number of available spaces for:

- Big cars
- Medium cars
- Small cars

The addCar() method checks whether a suitable parking space is available.

If a space is available, decrease the corresponding count and return true.

Otherwise, return false.

JAVA CODE

```
class ParkingSystem {
```

```
int big;
```

```
int medium;
```

```
int small;
```

```
public ParkingSystem(int big, int medium, int small) {
```

```

    this.big = big;
    this.medium = medium;
    this.small = small;
}

public boolean addCar(int carType) {

    if (carType == 1) {

        if (big > 0) {
            big--;
            return true;
        }

    } else if (carType == 2) {

        if (medium > 0) {
            medium--;
            return true;
        }

    } else {

        if (small > 0) {
            small--;
            return true;
        }

    }

    return false;
}
}

```

COMPLEXITY

Time Complexity: $O(1)$

Space Complexity: $O(1)$

PROBLEM 2: JAVA INSTANCEOF KEYWORD

Platform: HackerRank

Aim:

To understand inheritance and determine the type of an object using the instanceof operator.

Syntax:

object instanceof ClassName

The result is either true or false.

JAVA CODE

```
import java.util.*;
class Student {
}
class Rockstar {
}
class Hacker {
}
public class InstanceOFTutorial {
    static String count(ArrayList<Object> mylist) {

        int students = 0;
        int rockstars = 0;
        int hackers = 0;

        for (Object obj : mylist) {

            if (obj instanceof Student) {
                students++;
            }

            if (obj instanceof Rockstar) {
                rockstars++;
            }

            if (obj instanceof Hacker) {
                hackers++;
            }
        }

        return students + " " + rockstars + " " + hackers;
    }
}
```

}

EXPLANATION

The instanceof operator checks whether an object belongs to a particular class.

For example:

obj instanceof Student

returns true when obj is a Student object.

COMPLEXITY

Time Complexity: $O(n)$

Space Complexity: $O(1)$, excluding the input collection.

=====

SESSION 2

CLASS-TO-CLASS INTERACTION

OBJECTIVE

To understand how different classes interact with each other using object references.

Important Relationships:

- Association
- Aggregation
- Composition

Example:

Teacher ----- Student

A class can contain a reference to an object of another class.

PROBLEM 3: DESIGN UNDERGROUND SYSTEM

Platform: LeetCode

Problem Number: 1396

Aim:

To design a system that records passengers checking into and checking out of stations and calculates the average travel time.

Concepts Used:

- Classes
- Objects
- HashMap
- Class-to-class interaction

APPROACH

Use one HashMap to store current check-ins.

Each check-in stores:

- Customer ID
- Station
- Check-in time

Use another HashMap to store:

- Route
- Total travel time
- Number of trips

JAVA CODE

```
import java.util.*;

class UndergroundSystem {
    class CheckIn {

        String station;
        int time;

        CheckIn(String station, int time) {
            this.station = station;
            this.time = time;
        }
    }

    class Route {

        int totalTime;
        int count;

        Route(int totalTime, int count) {
            this.totalTime = totalTime;
            this.count = count;
        }
    }

    HashMap<Integer, CheckIn> checkIns;
    HashMap<String, Route> routes;

    public UndergroundSystem() {

        checkIns = new HashMap<>();
        routes = new HashMap<>();
    }

    public void checkIn(
        int id,
        String stationName,
```

```

        int t) {

    checkIns.put(
        id,
        new CheckIn(stationName, t)
    );
}

public void checkOut(
    int id,
    String stationName,
    int t) {

    CheckIn checkIn = checkIns.get(id);

    String key =
        checkIn.station + "->" + stationName;

    int travelTime =
        t - checkIn.time;

    if (!routes.containsKey(key)) {

        routes.put(
            key,
            new Route(travelTime, 1)
        );

    } else {

        Route route = routes.get(key);

        route.totalTime += travelTime;
        route.count++;
    }

    checkIns.remove(id);
}

public double getAverageTime(

```

```

        String startStation,
        String endStation) {

    String key =
        startStation + "->" + endStation;

    Route route = routes.get(key);

    return (double) route.totalTime
        / route.count;
}

```

```

}

```

COMPLEXITY

checkIn: $O(1)$ average

checkOut: $O(1)$ average

getAverageTime: $O(1)$ average

Space: $O(n)$

PROBLEM 4: DESIGN BROWSER HISTORY

Platform: LeetCode

Problem Number: 1472

Aim:

To design a browser history system supporting:

- Visit
- Back
- Forward

APPROACH

Use a doubly linked list.

Each node contains:

- URL
- Previous node
- Next node

JAVA CODE

```

class BrowserHistory {
    class Node {

```

```

        String url;
        Node prev;

```

```
Node next;
```

```
Node(String url) {  
    this.url = url;  
}  
}
```

```
Node current;
```

```
public BrowserHistory(String homepage) {  
  
    current = new Node(homepage);  
}
```

```
public void visit(String url) {  
  
    Node newNode = new Node(url);  
  
    current.next = newNode;  
    newNode.prev = current;  
  
    current = newNode;  
}
```

```
public String back(int steps) {  
  
    while (steps > 0 &&  
        current.prev != null) {  
  
        current = current.prev;  
        steps--;  
    }  
  
    return current.url;  
}
```

```
public String forward(int steps) {  
  
    while (steps > 0 &&  
        current.next != null) {
```

```

        current = current.next;
        steps--;
    }

    return current.url;
}

```

}
 COMPLEXITY

Visit: $O(1)$

Back: $O(\text{steps})$

Forward: $O(\text{steps})$

Space: $O(n)$

=====

SESSION 3

CLASS-TO-METHOD INTERACTION

OBJECTIVE

To understand how methods operate on the data stored inside objects.

A class generally contains:

Data + Methods

PROBLEM 5: DESIGN HASHSET

Platform: LeetCode

Problem Number: 705

Aim:

To design a HashSet without using Java's built-in HashSet class.

Required Operations:

- add(key)
- remove(key)
- contains(key)

APPROACH

For the given key range, use a boolean array.

true = key exists

false = key does not exist

JAVA CODE

```

class MyHashSet {
    boolean[] set;
}

```

```

public MyHashSet() {

    set = new boolean[1000001];
}

public void add(int key) {

    set[key] = true;
}

public void remove(int key) {

    set[key] = false;
}

public boolean contains(int key) {

    return set[key];
}

```

}
 COMPLEXITY

Add: $O(1)$

Remove: $O(1)$

Contains: $O(1)$

Space: $O(k)$

PROBLEM 6: GRADING STUDENTS

Platform: HackerRank

Aim:

To round student grades according to the specified rules.

RULES

If the grade is less than 38, do not round it.

Find the next multiple of 5.

If the difference is less than 3, round the grade.

Otherwise, keep the original grade.

Example:

Grade = 84

Next multiple = 85

Difference = 1

Result = 85

JAVA CODE

```
import java.util.*;
```

```
class Result {
```

```
    public static List<Integer> gradingStudents(  
        List<Integer> grades) {
```

```
        List<Integer> result =  
            new ArrayList<>();
```

```
        for (int grade : grades) {
```

```
            if (grade < 38) {
```

```
                result.add(grade);
```

```
            } else {
```

```
                int nextMultiple =  
                    ((grade / 5) + 1) * 5;
```

```
                if (nextMultiple - grade < 3) {
```

```
                    result.add(nextMultiple);
```

```
                } else {
```

```
                    result.add(grade);
```

```
                }
```

```
            }
```

```
        }
```

```
        return result;
```

```
    }
```

```
}
```

COMPLEXITY

Time Complexity: O(n)

Space Complexity: $O(n)$ for the result list

=====

SESSION 4

INHERITANCE

OBJECTIVE

To understand inheritance and how a child class can reuse the properties and methods of a parent class.

Syntax:

class Child extends Parent

Parent class = Superclass

Child class = Subclass

PROBLEM 7: JAVA INHERITANCE 1

Platform: HackerRank

Problem: Java Inheritance 1

Aim:

To demonstrate basic inheritance in Java.

JAVA CODE

```
class Animal {
```

```
void walk() {
```

```
    System.out.println(
```

```
        "I am walking"
```

```
    );
```

```
}
```

```
}
```

```
class Bird extends Animal {
```

```
void fly() {
```

```
    System.out.println(
```

```
        "I am flying"
```

```
    );
```

```
}
```

```
void sing() {
```

```
    System.out.println(
```

```
        "I am singing"
```

```

    );
}

}

public class Solution {
    public static void main(String[] args) {

        Bird bird = new Bird();

        bird.walk();
        bird.fly();
        bird.sing();
    }
}

```

}
 OUTPUT

I am walking
 I am flying
 I am singing

EXPLANATION

Bird extends Animal.

Therefore, a Bird object can access:

walk() - inherited from Animal

fly() - defined in Bird

sing() - defined in Bird

COMPLEXITY

Time Complexity: $O(1)$

Space Complexity: $O(1)$

PROBLEM 8: JAVA INHERITANCE 2

Platform: HackerRank

Problem: Java Inheritance 2

Aim:

To demonstrate inheritance and the use of inherited methods.

JAVA CODE

```

class Arithmetic {
    public int add(int a, int b) {

```

```

        return a + b;
    }

}

class Adder extends Arithmetic {
}

public class Solution {
    public static void main(String[] args) {

        Adder a = new Adder();

        System.out.println(
            "My superclass is: " +
            a.getClass()
            .getSuperclass()
            .getName()
        );

        System.out.println(
            a.add(10, 32) + " " +
            a.add(10, 3) + " " +
            a.add(10, 10)
        );
    }
}

```

```

}

```

OUTPUT

My superclass is: Arithmetic

42 13 20

EXPLANATION

Adder extends Arithmetic.

Therefore, an Adder object inherits the add() method from Arithmetic.

COMPLEXITY

Time Complexity: $O(1)$

Space Complexity: $O(1)$

=====

SESSION 5

DESIGNING REUSABLE CLASSES

OBJECTIVE

To understand how reusable classes and methods can be designed using Java Generics and object-oriented structures.

PROBLEM 9: JAVA GENERICS

Platform: HackerRank

Problem: Java Generics

Aim:

To understand how Java Generics allow the same method to work with different data types.

Generic method syntax:

```
public <T> void printArray(T[] array)
```

JAVA CODE

```
import java.util.*;

public class Printer {

    public <T> void printArray(T[] array) {

        for (T element : array) {

            System.out.println(element);
        }
    }
}
```

```
}
```

EXAMPLE

```
Integer[] intArray = {1, 2, 3};
```

```
String[] stringArray = {
```

```
"Hello",
```

```
"World"
```

```
};
```

EXPLANATION

<T> is a type parameter.

The same method can work with different types such as:

- Integer
- String
- Double
- Character

COMPLEXITY

Time Complexity: $O(n)$

Extra Space: $O(1)$

PROBLEM 10: THRONE INHERITANCE

Platform: LeetCode

Problem Number: 1600

Aim:

To design a system that maintains the order of succession in a royal family.

Operations:

- Birth
- Death
- Get inheritance order

CONCEPTS USED

- Classes
- Objects
- HashMap
- HashSet
- ArrayList
- Tree
- DFS

APPROACH

Represent every person as a node in a family tree.

Store:

Person $\rightarrow$ List of children

Maintain a HashSet containing people who have died.

To generate the inheritance order:

Start with the king.

Add the person if they are alive.

Visit each child in birth order.

Continue recursively.

JAVA CODE

```
import java.util.*;
```

```
class ThroneInheritance {
```

```
    String king;
```

```
    HashMap<String, List<String>> children;
```

```
    HashSet<String> dead;
```

```
    public ThroneInheritance(  
        String kingName) {
```

```

king = kingName;

children = new HashMap<>();

dead = new HashSet<>();

children.put(
    kingName,
    new ArrayList<>()
);
}

public void birth(
    String parentName,
    String childName) {

    children
        .computeIfAbsent(
            parentName,
            k -> new ArrayList<>()
        )
        .add(childName);

    children.putIfAbsent(
        childName,
        new ArrayList<>()
    );
}

public void death(String name) {

    dead.add(name);
}

public List<String>
getInheritanceOrder() {

    List<String> result =
        new ArrayList<>();

```

```

        dfs(king, result);

        return result;
    }

    private void dfs(
        String person,
        List<String> result) {

        if (!dead.contains(person)) {

            result.add(person);
        }

        for (String child :
            children.get(person)) {

            dfs(child, result);
        }
    }
}

```

}

COMPLEXITY

Birth: $O(1)$ average

Death: $O(1)$ average

Get Inheritance Order: $O(n)$

Space: $O(n)$

PROBLEM 1: DESIGN PARKING SYSTEM

Platform: LeetCode

Problem: 1603. Design Parking System

Aim:

To design a parking system that maintains the number of available spaces for big, medium, and small cars.

Concepts Used:

- Class
- Object

- Constructor
- Instance variables
- Methods

Example:

Input:

ParkingSystem(1, 1, 0)

addCar(1)

addCar(2)

addCar(3)

addCar(1)

Output:

true

true

false

false

Approach:

Create a ParkingSystem class.

Store the available number of:

- Big spaces
- Medium spaces
- Small spaces

The addCar() method checks whether a suitable space is available.

If available, decrease the corresponding count and return true.

Otherwise, return false.

JAVA CODE

```
class ParkingSystem {
```

```
int big;
```

```
int medium;
```

```
int small;
```

```
public ParkingSystem(int big, int medium, int small) {
```

```
    this.big = big;
```

```
    this.medium = medium;
```

```
    this.small = small;
```

```
}
```

```
public boolean addCar(int carType) {
```

```
    if (carType == 1) {
```

```

        if (big > 0) {
            big--;
            return true;
        }

    } else if (carType == 2) {

        if (medium > 0) {
            medium--;
            return true;
        }

    } else {

        if (small > 0) {
            small--;
            return true;
        }

    }

    return false;
}

}

```

Complexity:

Time: $O(1)$ per operation

Space: $O(1)$

PROBLEM 2: JAVA INSTANCEOF KEYWORD

Platform: HackerRank

Problem: Java Instanceof Keyword

Aim:

To understand inheritance and determine whether an object belongs to a particular class using the instanceof operator.

Syntax:

object instanceof ClassName

The result is either true or false.

JAVA CODE

```
import java.util.*;
```

```

class Student {
}
class Rockstar {
}
class Hacker {
}
public class InstanceOFTutorial {
static String count(ArrayList<Object> mylist) {

    int students = 0;
    int rockstars = 0;
    int hackers = 0;

    for (Object obj : mylist) {

        if (obj instanceof Student) {
            students++;
        }

        if (obj instanceof Rockstar) {
            rockstars++;
        }

        if (obj instanceof Hacker) {
            hackers++;
        }
    }

    return students + " " + rockstars + " " + hackers;
}

}

```

Explanation:

The instanceof operator checks the type of an object.

Example:

obj instanceof Student

returns true if obj is an instance of Student.

Complexity:

Time: $O(n)$

Space: $O(1)$, excluding the input collection

=====

SESSION 2

CLASS-TO-CLASS INTERACTION

OBJECTIVE

To understand how different classes interact with each other using object references and relationships.

Important Relationships:

- Association
- Aggregation
- Composition

Example:

Teacher ---- Student

A class can contain a reference to another class.

PROBLEM 3: DESIGN UNDERGROUND SYSTEM

Platform: LeetCode

Problem: 1396. Design Underground System

Aim:

To design a system that records passengers checking into and checking out of stations and calculates the average travel time.

Concepts Used:

- Classes
- Objects
- HashMap
- Class-to-class interaction

Approach:

Use one HashMap to store current check-ins.

Store:

Customer ID

Station

Check-in time

Use another HashMap to store:

Route

Total travel time

Number of trips

JAVA CODE

```
import java.util.*;  
class UndergroundSystem {  
    class CheckIn {
```

```

String station;
int time;

    CheckIn(String station, int time) {
        this.station = station;
        this.time = time;
    }
}

class Route {

    int totalTime;
    int count;

    Route(int totalTime, int count) {
        this.totalTime = totalTime;
        this.count = count;
    }
}

HashMap<Integer, CheckIn> checkIns;
HashMap<String, Route> routes;

public UndergroundSystem() {

    checkIns = new HashMap<>();
    routes = new HashMap<>();
}

public void checkIn(int id, String stationName, int t) {

    checkIns.put(id, new CheckIn(stationName, t));
}

public void checkOut(int id, String stationName, int t) {

    CheckIn checkIn = checkIns.get(id);

    String key = checkIn.station + "->" + stationName;

```

```

int travelTime = t - checkIn.time;

if (!routes.containsKey(key)) {

    routes.put(key, new Route(travelTime, 1));

} else {

    Route route = routes.get(key);

    route.totalTime += travelTime;
    route.count++;
}

checkIns.remove(id);
}

public double getAverageTime(
    String startStation,
    String endStation) {

    String key = startStation + "->" + endStation;

    Route route = routes.get(key);

    return (double) route.totalTime / route.count;
}

```

```

}

```

Complexity:

checkIn: $O(1)$ average

checkOut: $O(1)$ average

getAverageTime: $O(1)$ average

Space: $O(n)$

PROBLEM 4: DESIGN BROWSER HISTORY

Platform: LeetCode

Problem: 1472. Design Browser History

Aim:

To design a browser history system that supports:

- Visit
- Back
- Forward

Approach:

Use a doubly linked list.

Each node contains:

URL

Previous node

Next node

JAVA CODE

```
class BrowserHistory {
```

```
class Node {
```

```
    String url;
```

```
    Node prev;
```

```
    Node next;
```

```
    Node(String url) {
```

```
        this.url = url;
```

```
    }
```

```
}
```

```
Node current;
```

```
public BrowserHistory(String homepage) {
```

```
    current = new Node(homepage);
```

```
}
```

```
public void visit(String url) {
```

```
    Node newNode = new Node(url);
```

```
    current.next = newNode;
```

```
    newNode.prev = current;
```

```
    current = newNode;
```

```
}
```

```

public String back(int steps) {

    while (steps > 0 && current.prev != null) {

        current = current.prev;
        steps--;
    }

    return current.url;
}

```

```

public String forward(int steps) {

    while (steps > 0 && current.next != null) {

        current = current.next;
        steps--;
    }

    return current.url;
}

```

```

}
Complexity:
Visit: O(1)
Back: O(steps)
Forward: O(steps)
Space: O(n)

```

=====

SESSION 3

CLASS-TO-METHOD INTERACTION

OBJECTIVE

To understand how methods operate on the data stored inside objects.

A class generally contains:

Data + Methods

PROBLEM 5: DESIGN HASHSET

Platform: LeetCode

Problem: 705. Design HashSet

Aim:

To design a HashSet without using Java's built-in HashSet class.

Required Operations:

- add(key)
- remove(key)
- contains(key)

Approach:

For the given key range, use a boolean array.

true = key exists

false = key does not exist

JAVA CODE

```
class MyHashSet {
```

```
    boolean[] set;
```

```
    public MyHashSet() {
```

```
        set = new boolean[1000001];
    }
```

```
    public void add(int key) {
```

```
        set[key] = true;
    }
```

```
    public void remove(int key) {
```

```
        set[key] = false;
    }
```

```
    public boolean contains(int key) {
```

```
        return set[key];
    }
```

```
}
```

Complexity:

Add: O(1)

Remove: O(1)

Contains: $O(1)$

Space: $O(k)$

PROBLEM 6: GRADING STUDENTS

Platform: HackerRank

Problem: Grading Students

Aim:

To round student grades according to the given rules.

Rules:

If the grade is less than 38, do not round it.

Find the next multiple of 5.

If the difference between the grade and the next multiple is less than 3, round the grade.

Otherwise, keep the original grade.

Example:

Grade = 84

Next multiple = 85

Difference = 1

Result = 85

JAVA CODE

```
import java.util.*;
class Result {
    public static List<Integer> gradingStudents(
        List<Integer> grades) {

        List<Integer> result = new ArrayList<>();

        for (int grade : grades) {

            if (grade < 38) {

                result.add(grade);

            } else {

                int nextMultiple =
                    ((grade / 5) + 1) * 5;

                if (nextMultiple - grade < 3) {
                    result.add(nextMultiple);
                } else {
```

```

        result.add(grade);
    }
}

return result;
}

```

```

}

```

Complexity:

Time: $O(n)$

Space: $O(n)$ for the result list

=====

SESSION 4

INHERITANCE

OBJECTIVE

To understand inheritance and how a child class can reuse properties and methods of a parent class.

Syntax:

class Child extends Parent

Parent class = Superclass

Child class = Subclass

PROBLEM 7: JAVA INHERITANCE 1

Platform: HackerRank

Problem: Java Inheritance 1

Aim:

To demonstrate basic inheritance in Java.

JAVA CODE

```

class Animal {
void walk() {
    System.out.println("I am walking");
}
}

```

```

}

```

```

class Bird extends Animal {

```

```

void fly() {
    System.out.println("I am flying");
}
}

```

```

    }

    void sing() {
        System.out.println("I am singing");
    }

}

public class Solution {
    public static void main(String[] args) {

        Bird bird = new Bird();

        bird.walk();
        bird.fly();
        bird.sing();
    }

}

```

}
 Output:
 I am walking
 I am flying
 I am singing

Explanation:
 Bird extends Animal.
 Therefore, a Bird object can use:
 walk() - inherited from Animal
 fly() - defined in Bird
 sing() - defined in Bird

Complexity:
 Time: $O(1)$
 Space: $O(1)$

PROBLEM 8: JAVA INHERITANCE 2

Platform: HackerRank

Problem: Java Inheritance 2

Aim:

To demonstrate inheritance and the use of inherited methods.

JAVA CODE

```

class Arithmetic {
public int add(int a, int b) {
    return a + b;
}

}

class Adder extends Arithmetic {
}

public class Solution {
public static void main(String[] args) {

    Adder a = new Adder();

    System.out.println(
        "My superclass is: " +
        a.getClass().getSuperclass().getName()
    );

    System.out.println(
        a.add(10, 32) + " " +
        a.add(10, 3) + " " +
        a.add(10, 10)
    );
}
}

```

}

Output:

My superclass is: Arithmetic

42 13 20

Explanation:

Adder extends Arithmetic.

Therefore, Adder inherits the add() method from Arithmetic.

Complexity:

Time: O(1)

Space: O(1)

=====

SESSION 5

DESIGNING REUSABLE CLASSES

OBJECTIVE

To understand how reusable classes and methods can be designed using Java Generics and object-oriented structures.

PROBLEM 9: JAVA GENERICS

Platform: HackerRank

Problem: Java Generics

Aim:

To understand how Java Generics allow the same method to work with different data types.

Generic method syntax:

```
public <T> void printArray(T[] array)
```

JAVA CODE

```
import java.util.*;

public class Printer {

    public <T> void printArray(T[] array) {

        for (T element : array) {

            System.out.println(element);
        }
    }
}
```

```
}
```

Example:

```
Integer[] intArray = {1, 2, 3};
```

```
String[] stringArray = {
```

```
"Hello",
```

```
"World"
```

```
};
```

Explanation:

<T> is a type parameter.

The same method can work with different types such as:

Integer

String

Double

Character

Complexity:

Time: O(n)

Extra Space: $O(1)$

PROBLEM 10: THRONE INHERITANCE

Platform: LeetCode

Problem: 1600. Throne Inheritance

Aim:

To design a system that maintains the order of succession in a royal family.

Operations:

- Birth
- Death
- Get inheritance order

Concepts Used:

- Classes
- Objects
- HashMap
- HashSet
- ArrayList
- Tree
- DFS

Approach:

Represent every person as a node in a family tree.

Store:

Person -> List of children

Maintain a HashSet containing people who have died.

To generate the inheritance order:

Start with the king.

Add the person if they are alive.

Visit each child in birth order.

Continue recursively.

JAVA CODE

```
import java.util.*;
```

```
class ThroneInheritance {
```

```
    String king;
```

```
    HashMap<String, List<String>> children;
```

```
    HashSet<String> dead;
```

```
    public ThroneInheritance(String kingName) {
```

```
king = kingName;
```

```
children = new HashMap<>();
```

```
dead = new HashSet<>();
```

```
children.put(  
    kingName,  
    new ArrayList<>()
```

```
);
```

```
}
```

```
public void birth(  
    String parentName,  
    String childName) {
```

```
    children  
        .computeIfAbsent(  
            parentName,  
            k -> new ArrayList<>()
```

```
        )
```

```
        .add(childName);
```

```
    children.putIfAbsent(  
        childName,  
        new ArrayList<>()
```

```
    );
```

```
}
```

```
public void death(String name) {
```

```
    dead.add(name);
```

```
}
```

```
public List<String> getInheritanceOrder() {
```

```
    List<String> result =  
        new ArrayList<>();
```

```
    dfs(king, result);
```



```

        return result;
    }

    private void dfs(
        String person,
        List<String> result) {

        if (!dead.contains(person)) {

            result.add(person);
        }

        for (String child :
            children.get(person)) {

            dfs(child, result);
        }
    }
}

```

}

Complexity:

Birth: $O(1)$ average

Death: $O(1)$ average

Get Inheritance Order: $O(n)$

Space: $O(n)$

=====

QUICK REVISION TABLE

Design Parking System

Concept: Classes and Objects

Time: $O(1)$

Space: $O(1)$

Java Instanceof Keyword

Concept: Object Type Checking

Time: $O(n)$

Space: $O(1)$

Design Underground System

Concept: Class Interaction + HashMap

Time: $O(1)$ average

Space: $O(n)$

Design Browser History

Concept: Doubly Linked List + Objects

Time: $O(\text{steps})$

Space: $O(n)$

Design HashSet

Concept: Class Design

Time: $O(1)$

Space: $O(k)$

Grading Students

Concept: Methods + ArrayList

Time: $O(n)$

Space: $O(n)$

Java Inheritance 1

Concept: Inheritance

Time: $O(1)$

Space: $O(1)$

Java Inheritance 2

Concept: Inheritance

Time: $O(1)$

Space: $O(1)$

Java Generics

Concept: Generic Methods

Time: $O(n)$

Space: $O(1)$

Throne Inheritance

Concept: Tree + DFS + OOP

Time: $O(n)$

Space: $O(n)$

=====

IMPORTANT JAVA OOP CONCEPTS

CLASS

A class is a blueprint for creating objects.

Example:

```
class Student {
```

```
String name;
```

```
int age;
```

```
}
```

OBJECT

An object is an instance of a class.

Example:

```
Student s = new Student();
```

ENCAPSULATION

Encapsulation means combining data and methods inside a class and controlling access to the data.

Example:

```
class Student {  
    private int age;
```

```
    public void setAge(int age) {  
        this.age = age;  
    }  
  
    public int getAge() {  
        return age;  
    }  
}
```

```
}
```

INHERITANCE

Inheritance allows a child class to reuse functionality from a parent class.

Example:

```
class Dog extends Animal {  
}
```

POLYMORPHISM

Polymorphism allows the same method or interface to behave differently depending on the object.

Example:

```
Animal a = new Dog();
```

ABSTRACTION

Abstraction hides implementation details and exposes only the required functionality.

It can be implemented using:

- Abstract classes
- Interfaces

ASSOCIATION

Association represents a relationship between two independent classes.

Example:

Teacher ---- Student

AGGREGATION

Aggregation represents a weak "has-a" relationship.
The contained object can exist independently.

Example:

Department ---- Teacher

COMPOSITION

Composition represents a strong "has-a" relationship.

Example:

House ---- Room

GENERICIS

Generics allow classes and methods to work with different data types.

Example:

```
class Box<T> {
```

```
    T value;
```

```
}
```

```
=====
```

FINAL EXAM CHECKLIST

Before the examination, make sure you can explain:

- ☐ What is a class?
- ☐ What is an object?
- ☐ Difference between class and object
- ☐ What is encapsulation?
- ☐ What is inheritance?
- ☐ What is polymorphism?
- ☐ What is abstraction?
- ☐ Difference between association, aggregation, and composition
- ☐ How instanceof works
- ☐ How classes interact with each other
- ☐ How objects interact through methods
- ☐ How to design reusable classes
- ☐ What are Java Generics?
- ☐ How to use a generic method
- ☐ How inheritance works using extends
- ☐ How to implement a tree using classes
- ☐ How DFS is used in object-oriented problems

```
=====
```

UNIT III SUMMARY

Object-Oriented Problem Solving combines DSA and OOP concepts.

The general problem-solving process is:

Identify Objects

↓

Create Classes

↓

Define Data

↓

Define Methods

↓

Establish Class Relationships

↓

Apply Data Structures

↓

Solve the Problem

=====

END OF UNIT III

Object-Oriented Problem Solving

Week 7 | Sessions 1–5 | 10 Hands-On Problems

You said:

PDF FRIENDLY

ChatGPT said:

Yes — PDF-friendly means no dark code boxes, no Markdown styling, and no colored background.

Use this exact setup when pasting into Word / Google Docs:

PDF Page Settings

Page: A4

Orientation: Portrait

Margins: Normal (1 inch)

Background: White

Body font: Times New Roman, 12 pt

Main headings: 18 pt, Bold, Black

Subheadings: 14 pt, Bold, Black

Code: Consolas, 10 pt, Black text

Code background: White

Line spacing: 1.15

Paragraph spacing: 6 pt

Page numbers: Bottom center

Important

Do not copy the code from a dark code block. Copy the plain-text document version above into Word/Google Docs, then apply the settings above.

For the cleanest PDF, use:

Word:

File → Save As → PDF

Google Docs:

File → Download → PDF Document (.pdf)

If you want, I can also give you a fully page-by-page A4 layout, with each problem starting cleanly on the appropriate page so you can paste it directly into Word and export without any black background.

You said:

GIVE AGAIN

ChatGPT said:

Yes. Here is the PDF-FRIENDLY VERSION AGAIN — designed for white A4 pages, with black text only and no dark code backgrounds.

Copy everything below into Microsoft Word or Google Docs, then export as PDF.

UNIT III – OBJECT-ORIENTED PROBLEM SOLVING

Course Outcome:

Design object-oriented structures for problem solving.

WEEK 7

=====

SESSION 1

INTRODUCTION TO OBJECT MODELING IN ALGORITHMIC PROBLEMS

=====

OBJECTIVE

To understand how real-world entities can be represented using classes and objects and how object-oriented structures can be used to solve algorithmic problems.

Important Concepts:

- Classes
- Objects
- Constructors
- Instance variables
- Methods
- Object modeling

PROBLEM 1: DESIGN PARKING SYSTEM

Platform: LeetCode

Problem Number: 1603

Aim:

To design a parking system that maintains the number of available spaces for big, medium, and small cars.

Vehicle Types:

Big

Medium

Small

Example:

Input:

ParkingSystem(1, 1, 0)

addCar(1)

addCar(2)

addCar(3)

addCar(1)

Output:

true

true

false

false

Approach:

Create a ParkingSystem class.

Store the number of available spaces for:

- Big cars
- Medium cars
- Small cars

The addCar() method checks whether a suitable parking space is available.

If a space is available, decrease the corresponding count and return true.

Otherwise, return false.

JAVA CODE

```
class ParkingSystem {
```

```
    int big;
```

```
    int medium;
```

```
    int small;
```

```
    public ParkingSystem(int big, int medium, int small) {
```

```
        this.big = big;
```

```
        this.medium = medium;
```

```
        this.small = small;
```

```
    }
```

```
    public boolean addCar(int carType) {
```

```
        if (carType == 1) {
```

```
            if (big > 0) {
```

```

        big--;
        return true;
    }

    } else if (carType == 2) {

        if (medium > 0) {
            medium--;
            return true;
        }

    } else {

        if (small > 0) {
            small--;
            return true;
        }
    }

    return false;
}

```

}

COMPLEXITY

Time Complexity: $O(1)$

Space Complexity: $O(1)$

PROBLEM 2: JAVA INSTANCEOF KEYWORD

Platform: HackerRank

Aim:

To understand inheritance and determine the type of an object using the instanceof operator.

Syntax:

object instanceof ClassName

The result is either true or false.

JAVA CODE

```
import java.util.*;
```

```
class Student {
```

```
}
```



```

class Rockstar {
}
class Hacker {
}
public class InstanceOFTutorial {
static String count(ArrayList<Object> mylist) {

    int students = 0;
    int rockstars = 0;
    int hackers = 0;

    for (Object obj : mylist) {

        if (obj instanceof Student) {
            students++;
        }

        if (obj instanceof Rockstar) {
            rockstars++;
        }

        if (obj instanceof Hacker) {
            hackers++;
        }
    }

    return students + " " + rockstars + " " + hackers;
}
}

```

EXPLANATION

The instanceof operator checks whether an object belongs to a particular class.

For example:

obj instanceof Student

returns true when obj is a Student object.

COMPLEXITY

Time Complexity: $O(n)$

Space Complexity: $O(1)$, excluding the input collection.

=====

SESSION 2

CLASS-TO-CLASS INTERACTION

OBJECTIVE

To understand how different classes interact with each other using object references.

Important Relationships:

- Association
- Aggregation
- Composition

Example:

Teacher ----- Student

A class can contain a reference to an object of another class.

PROBLEM 3: DESIGN UNDERGROUND SYSTEM

Platform: LeetCode

Problem Number: 1396

Aim:

To design a system that records passengers checking into and checking out of stations and calculates the average travel time.

Concepts Used:

- Classes
- Objects
- HashMap
- Class-to-class interaction

APPROACH

Use one HashMap to store current check-ins.

Each check-in stores:

- Customer ID
- Station
- Check-in time

Use another HashMap to store:

- Route
- Total travel time
- Number of trips

JAVA CODE

```
import java.util.*;  
class UndergroundSystem {  
    class CheckIn {
```

```
        String station;  
        int time;
```

```
    CheckIn(String station, int time) {  
        this.station = station;  
        this.time = time;  
    }  
}
```

```
class Route {
```

```
    int totalTime;  
    int count;
```

```
    Route(int totalTime, int count) {  
        this.totalTime = totalTime;  
        this.count = count;  
    }  
}
```

```
HashMap<Integer, CheckIn> checkIns;  
HashMap<String, Route> routes;
```

```
public UndergroundSystem() {
```

```
    checkIns = new HashMap<>();  
    routes = new HashMap<>();  
}
```

```
public void checkIn(  
    int id,  
    String stationName,  
    int t) {
```

```
    checkIns.put(  
        id,  
        new CheckIn(stationName, t)  
    );  
}
```

```
public void checkOut(  
    int id,
```

```

        String stationName,
        int t) {

    CheckIn checkIn = checkIns.get(id);

    String key =
        checkIn.station + "->" + stationName;

    int travelTime =
        t - checkIn.time;

    if (!routes.containsKey(key)) {

        routes.put(
            key,
            new Route(travelTime, 1)
        );

    } else {

        Route route = routes.get(key);

        route.totalTime += travelTime;
        route.count++;
    }

    checkIns.remove(id);
}

public double getAverageTime(
    String startStation,
    String endStation) {

    String key =
        startStation + "->" + endStation;

    Route route = routes.get(key);

    return (double) route.totalTime
        / route.count;
}

```

```
}
```

```
}
```

COMPLEXITY

checkIn: $O(1)$ average

checkOut: $O(1)$ average

getAverageTime: $O(1)$ average

Space: $O(n)$

PROBLEM 4: DESIGN BROWSER HISTORY

Platform: LeetCode

Problem Number: 1472

Aim:

To design a browser history system supporting:

- Visit
- Back
- Forward

APPROACH

Use a doubly linked list.

Each node contains:

- URL
- Previous node
- Next node

JAVA CODE

```
class BrowserHistory {  
    class Node {
```

```
        String url;  
        Node prev;  
        Node next;
```

```
        Node(String url) {  
            this.url = url;  
        }  
    }
```

```
}
```

```
    Node current;
```

```
    public BrowserHistory(String homepage) {
```

```
    current = new Node(homepage);
}

public void visit(String url) {

    Node newNode = new Node(url);

    current.next = newNode;
    newNode.prev = current;

    current = newNode;
}

public String back(int steps) {

    while (steps > 0 &&
           current.prev != null) {

        current = current.prev;
        steps--;
    }

    return current.url;
}

public String forward(int steps) {

    while (steps > 0 &&
           current.next != null) {

        current = current.next;
        steps--;
    }

    return current.url;
}

}
```

COMPLEXITY

Visit: $O(1)$

Back: $O(\text{steps})$

Forward: $O(\text{steps})$

Space: $O(n)$

=====

SESSION 3

CLASS-TO-METHOD INTERACTION

OBJECTIVE

To understand how methods operate on the data stored inside objects.

A class generally contains:

Data + Methods

PROBLEM 5: DESIGN HASHSET

Platform: LeetCode

Problem Number: 705

Aim:

To design a HashSet without using Java's built-in HashSet class.

Required Operations:

- add(key)
- remove(key)
- contains(key)

APPROACH

For the given key range, use a boolean array.

true = key exists

false = key does not exist

JAVA CODE

```
class MyHashSet {  
    boolean[] set;
```

```
    public MyHashSet() {
```

```
        set = new boolean[1000001];  
    }
```

```
    public void add(int key) {
```

```
        set[key] = true;  
    }
```

```

public void remove(int key) {

    set[key] = false;
}

public boolean contains(int key) {

    return set[key];
}

}

```

COMPLEXITY

Add: $O(1)$

Remove: $O(1)$

Contains: $O(1)$

Space: $O(k)$

PROBLEM 6: GRADING STUDENTS

Platform: HackerRank

Aim:

To round student grades according to the specified rules.

RULES

If the grade is less than 38, do not round it.

Find the next multiple of 5.

If the difference is less than 3, round the grade.

Otherwise, keep the original grade.

Example:

Grade = 84

Next multiple = 85

Difference = 1

Result = 85

JAVA CODE

```

import java.util.*;

class Result {

    public static List<Integer> gradingStudents(
        List<Integer> grades) {

        List<Integer> result =
            new ArrayList<>();
    }
}

```



```

for (int grade : grades) {

    if (grade < 38) {

        result.add(grade);

    } else {

        int nextMultiple =
            ((grade / 5) + 1) * 5;

        if (nextMultiple - grade < 3) {

            result.add(nextMultiple);

        } else {

            result.add(grade);
        }
    }
}

return result;
}

```

}

COMPLEXITY

Time Complexity: $O(n)$

Space Complexity: $O(n)$ for the result list

=====

SESSION 4

INHERITANCE

OBJECTIVE

To understand inheritance and how a child class can reuse the properties and methods of a parent class.

Syntax:

class Child extends Parent

Parent class = Superclass

Child class = Subclass

PROBLEM 7: JAVA INHERITANCE 1

Platform: HackerRank

Problem: Java Inheritance 1

Aim:

To demonstrate basic inheritance in Java.

JAVA CODE

```
class Animal {
    void walk() {

        System.out.println(
            "I am walking"
        );
    }

}

class Bird extends Animal {
    void fly() {

        System.out.println(
            "I am flying"
        );
    }

    void sing() {

        System.out.println(
            "I am singing"
        );
    }

}

public class Solution {
    public static void main(String[] args) {

        Bird bird = new Bird();
```

```
    bird.walk();
    bird.fly();
    bird.sing();
}
```

```
}
```

OUTPUT

I am walking

I am flying

I am singing

EXPLANATION

Bird extends Animal.

Therefore, a Bird object can access:

walk() - inherited from Animal

fly() - defined in Bird

sing() - defined in Bird

COMPLEXITY

Time Complexity: $O(1)$

Space Complexity: $O(1)$

PROBLEM 8: JAVA INHERITANCE 2

Platform: HackerRank

Problem: Java Inheritance 2

Aim:

To demonstrate inheritance and the use of inherited methods.

JAVA CODE

```
class Arithmetic {
    public int add(int a, int b) {

        return a + b;
    }

}

class Adder extends Arithmetic {
}

public class Solution {
    public static void main(String[] args) {
```

```

Adder a = new Adder();

System.out.println(
    "My superclass is: " +
    a.getClass()
    .getSuperclass()
    .getName()
);

System.out.println(
    a.add(10, 32) + " " +
    a.add(10, 3) + " " +
    a.add(10, 10)
);
}

```

```

}

```

OUTPUT

My superclass is: Arithmetic
42 13 20

EXPLANATION

Adder extends Arithmetic.

Therefore, an Adder object inherits the add() method from Arithmetic.

COMPLEXITY

Time Complexity: $O(1)$

Space Complexity: $O(1)$

=====

SESSION 5

DESIGNING REUSABLE CLASSES

OBJECTIVE

To understand how reusable classes and methods can be designed using Java Generics and object-oriented structures.

PROBLEM 9: JAVA GENERICS

Platform: HackerRank

Problem: Java Generics

Aim:

To understand how Java Generics allow the same method to work with different data types.

Generic method syntax:

```
public <T> void printArray(T[] array)
```

JAVA CODE

```
import java.util.*;
```

```
public class Printer {
```

```
    public <T> void printArray(T[] array) {
```

```
        for (T element : array) {
```

```
            System.out.println(element);
```

```
        }
```

```
    }
```

```
}
```

EXAMPLE

```
Integer[] intArray = {1, 2, 3};
```

```
String[] stringArray = {
```

```
"Hello",
```

```
"World"
```

```
};
```

EXPLANATION

<T> is a type parameter.

The same method can work with different types such as:

- Integer
- String
- Double
- Character

COMPLEXITY

Time Complexity: $O(n)$

Extra Space: $O(1)$

PROBLEM 10: THRONE INHERITANCE

Platform: LeetCode

Problem Number: 1600

Aim:

To design a system that maintains the order of succession in a royal family.

Operations:

- Birth
- Death

- Get inheritance order

CONCEPTS USED

- Classes
- Objects
- HashMap
- HashSet
- ArrayList
- Tree
- DFS

APPROACH

Represent every person as a node in a family tree.

Store:

Person → List of children

Maintain a HashSet containing people who have died.

To generate the inheritance order:

Start with the king.

Add the person if they are alive.

Visit each child in birth order.

Continue recursively.

JAVA CODE

```
import java.util.*;
```

```
class ThroneInheritance {
```

```
    String king;
```

```
    HashMap<String, List<String>> children;
```

```
    HashSet<String> dead;
```

```
    public ThroneInheritance(  
        String kingName) {
```

```
        king = kingName;
```

```
        children = new HashMap<>();
```

```
        dead = new HashSet<>();
```

```
        children.put(  
            kingName,  
            new ArrayList<>()
```

```
);  
}
```

```
public void birth(  
    String parentName,  
    String childName) {  
  
    children  
        .computeIfAbsent(  
            parentName,  
            k -> new ArrayList<>()  
        )  
        .add(childName);  
  
    children.putIfAbsent(  
        childName,  
        new ArrayList<>()  
    );  
}
```

```
public void death(String name) {  
  
    dead.add(name);  
}
```

```
public List<String>  
getInheritanceOrder() {  
  
    List<String> result =  
        new ArrayList<>();  
  
    dfs(king, result);  
  
    return result;  
}
```

```
private void dfs(  
    String person,  
    List<String> result) {
```

```
    if (!dead.contains(person)) {  
        result.add(person);  
    }  
  
    for (String child :  
        children.get(person)) {  
        dfs(child, result);  
    }  
}
```

```
}
```

COMPLEXITY

Birth: $O(1)$ average

Death: $O(1)$ average

Get Inheritance Order: $O(n)$

Space: $O(n)$